.NET (C#) & Vue 3 Project Setup Cheat Sheet

Quick-reference CLI commands, folder architecture, and connection boilerplate for full-stack development.

1. Prerequisites & Environment Check

Run these commands in your terminal to verify that your SDKs and runtimes are installed and accessible.

```
dotnet --version        # Verify .NET SDK (e.g., 9.0.x or 10.0.x)
node -v                 # Verify Node.js (LTS v20+ or v22+ recommended)
npm -v                  # Verify npm package manager
```

Tool / Extension	Purpose
VS Code + C# Dev Kit	Lightweight editor with full .NET debugging, IntelliSense, and NuGet support.
Vue - Official (Volar)	Essential VS Code extension for Vue 3 syntax highlighting and TypeScript/JS support.
Visual Studio 2022/2026	Full-featured IDE option for Windows users (select the *ASP.NET and web development* workload).

2. Recommended Monorepo Folder Structure

Keeping your backend API and frontend client inside one parent folder keeps version control clean and organized.

```
MyFullStackApp/
├── backend/                  # ASP.NET Core Web API (.NET)
│   ├── Controllers/          # API route endpoints
│   ├── Data/                 # EF Core DbContext & migrations
│   ├── Models/               # C# entity classes & DTOs
│   ├── Program.cs            # App startup, middleware & CORS setup
│   └── appsettings.json      # Database connection strings & config
└── frontend/                 # Vue 3 + Vite SPA
    ├── src/
    │   ├── assets/           # CSS, images, static files
    │   ├── components/       # Reusable UI pieces (Navbar, Cards)
    │   ├── router/           # Vue Router configuration
    │   ├── services/         # Axios / Fetch API call helpers
    │   ├── views/            # Page-level components (Home, Login)
    │   ├── App.vue           # Root Vue component
    │   └── main.js           # Vue app initialization
    └── package.json          # Frontend dependencies & scripts
```

3. Backend Setup: ASP.NET Core Web API

Execute these commands from your root `MyFullStackApp/` directory to scaffold and run the C# API.

```
# 1. Create a new Web API project with Controllers
dotnet new webapi -n backend --use-controllers
cd backend

# 2. Add essential Entity Framework Core & SQL packages (example: SQLite or SQL Server)
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Sqlite

# 3. Run the API with hot-reload (watch mode)
dotnet watch run
```

Note the local HTTPS/HTTP port printed in the terminal (for example, `http://localhost:5142`).

4. Frontend Setup: Vue 3 + Vite

Open a second terminal tab in `MyFullStackApp/` to create and run the Vue 3 client.

```
# 1. Scaffold a new Vue 3 project via Vite
npm create vue@latest frontend

# Recommended prompts when asked:
# ✔ Add TypeScript? No (choose Yes if you prefer strict typing)
# ✔ Add Vue Router for Single Page Application development? Yes
# ✔ Add Pinia for state management? Yes

# 2. Install dependencies, add Axios for HTTP calls, and start dev server
cd frontend
npm install
npm install axios
npm run dev
```

Vite typically starts your frontend at `http://localhost:5173`.

5. Connecting Vue 3 to .NET (CORS & API Calls)

Because Vue (`:5173`) and .NET (`:5142`) run on different ports during development, you must enable CORS in your backend.

Step A: Enable CORS in `backend/Program.cs`

```
var builder = WebApplication.CreateBuilder(args);

// 1. Add CORS policy allowing the Vue dev server
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowVueApp", policy =>
    {
        policy.WithOrigins("http://localhost:5173")
              .AllowAnyHeader()
              .AllowAnyMethod();
    });
});

builder.Services.AddControllers();
var app = builder.Build();

// 2. Apply CORS middleware BEFORE MapControllers
app.UseCors("AllowVueApp");
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Step B: Fetch Data in Vue 3 (`src/views/HomeView.vue`)

```
<script setup>
import { ref, onMounted } from 'vue';
import axios from 'axios';

const items = ref([]);
const error = ref('');

onMounted(async () => {
  try {
    const response = await axios.get('http://localhost:5142/api/items');
    items.value = response.data;
  } catch (err) {
    error.value = 'Failed to connect to .NET API.';
  }
});
</script>

<template>
  <p v-if="error">{{ error }}</p>
  <ul v-else>
    <li v-for="item in items" :key="item.id">{{ item.name }}</li>
  </ul>
</template>
```

6. Daily Terminal Cheat Sheet

Task	Command	Run Location
Create EF Core DB Migration	`dotnet ef migrations add InitialCreate`	`backend/`
Apply Migration to Database	`dotnet ef database update`	`backend/`
Install a NuGet Package	`dotnet add package <PackageName>`	`backend/`
Install an npm Package	`npm install <package-name>`	`frontend/`
Build Vue for Production	`npm run build`	`frontend/`